

立即体验

编程不再“断片”！Trae 智能记忆历史会话，一键继续， workflow 更连贯。

稀土掘金

NEW 首页

上新 AI Coding

沸点

课程

直播

活动

AI刷题

APP

插件

探索稀土掘金

创作者中心

会员

iOS：NSNotification.Name从OC到Swift的写法演进

season_zhu 2023-06-28 3,939 阅读4分钟 专栏：我的iOS开发

编程不再“断片”！Trae 智能记忆历史会话，一键继续， workflow 更连贯。

立即体验

前言

在闲来无事的时候，我会抽时间看看Foundation、UIKit等相关库的Swift代码说明与注释。说实话，有的时候看起来真的很乏味，也不容易理解。

不过有的时候也会觉得Apple这么设计API真是书写的简单漂亮，一脸佩服，今天给大家分享的就是从NSNotification.Name学习一种代码编写方式，并且已经在我自己的项目中进行类似这种写法的落地实战分享。

OC时代的通知名写法

NSNotification 想必大家都使用过，在iOS中处理跨非相邻页面、一对多的数据处理的时候，我们通常就是通过发通知传参，告之相关页面处理逻辑。

一般情况下，如果可以避免 **NSNotification** 的时候，我都会尽量避免使用，当然，既然系统给你了这个方法，那么在合适的场景使用也会妙手生花。

当然，对于 **NSNotification** 的通知名的管理，其实是一个看似简单，实际上可以做得非常优雅的事情。

特别是从OC过渡到Swift的过程，这段简单的代码，其实进行了很大的演变，我们不妨来看看。

下面这个是我早期写OC代码的时候，发送一个通知：

CSS

1 [[NSNotificationCenter defaultCenter] postNotificationName:@"CancelActivateSuccessNotification" obje

大家注意看，通知名，我就是非常简单的使用硬编码字符串 **@“CancelActivateSuccessNotification”** 来表示，硬编码的缺点就不用我多说了，编译器是不会给提示的，写错了，甚至连通知事件都没法收到，总之，这种写法是不好的。

于是，看看系统代码以及AFNetworking，我们会看见这样一种写法：

系统通知名：

objectivec

1 UIKIT_EXTERN NSNotificationName const UIApplicationDidFinishLaunchingNotification;

AFNetworking的通知名，也是学习系统通知名的写法进行的扩展：

season_zhu **LV.5**
大前端API调用工程师 @Otaku
优秀作者

134 文章

455k 阅读

785 粉丝

关注

私信

目录

收起

前言

OC时代的通知名写法

Swift时代的通知名写法

总结

吐槽

自己写的项目，欢迎大家star

- 相关推荐
- SwiftUI-动画
12k阅读 · 3点赞
- ByAI：Swift 中的不可复制类型
118阅读 · 0点赞
- Swift数据解析(第二篇) - Codable 上
2.9k阅读 · 13点赞
- 音视频学习笔记十三——渲染与滤镜之...
194阅读 · 0点赞
- 音视频学习笔记十——渲染与滤镜之iOS...
315阅读 · 2点赞

- 精选内容
- SwiftUI何时为值类型的视图提供持久标...
iOS开发小挖 · 10阅读 · 0点赞
- viewWillAppear与viewWillDisappear不...
songgeb · 31阅读 · 0点赞
- RxSwift：使用UITableViewCell的注意...
season_zhu · 42阅读 · 0点赞
- Swift 中强大的 Key Paths（键路径）机...
大熊猫侯佩 · 14阅读 · 0点赞
- Swift：Moya 中的MultiTarget详解
season_zhu · 85阅读 · 1点赞

.h文件

找对属于你的技术圈子

回复「进群」加入官方微信群



```
452     Posted when a task resumes.
453     */
454     FOUNDATION_EXPORT NSString * const AFNetworkingTaskDidResumeNotification;
455
456     /**
457     Posted when a task finishes executing. Includes a userInfo dictionary with additional information about the task.
458     */
459     FOUNDATION_EXPORT NSString * const AFNetworkingTaskDidCompleteNotification;
460
461     /**
462     Posted when a task suspends its execution.
463     */
464     FOUNDATION_EXPORT NSString * const AFNetworkingTaskDidSuspendNotification;
465
466     /**
467     Posted when a session is invalidated.
468     */
469     FOUNDATION_EXPORT NSString * const AFURLSessionDidInvalidateNotification;
470
471     /**
472     Posted when a session download task encountered an error when moving the temporary download file to a specified
473     destination.
474     */
475     FOUNDATION_EXPORT NSString * const AFURLSessionDownloadTaskDidFailToMoveFileNotification;
```

@稀土掘金技术社区

.m文件

```
72 NSString * const AFNetworkingTaskDidResumeNotification = @"com.alamofire.networking.task.resume";
73 NSString * const AFNetworkingTaskDidCompleteNotification = @"com.alamofire.networking.task.complete";
74 NSString * const AFNetworkingTaskDidSuspendNotification = @"com.alamofire.networking.task.suspend";
75 NSString * const AFURLSessionDidInvalidateNotification = @"com.alamofire.networking.session.invalidate";
76 NSString * const AFURLSessionDownloadTaskDidFailToMoveFileNotification =
    @"com.alamofire.networking.session.download.file-manager-error";
```

@稀土掘金技术社区

看起来并不是太高明？也许确实如此，只不过通过.h与.m的分隔，将一个硬编码字符串变成了一个全局可以引用、IDE可以快速键入的方式，但是它至少让调用变得简单与安全，这样就足够了。

于是乎，OC时代通知名的写法，我们基本上都会用以上这种方式进行编写：

.h

objective-c

[代码解读](#) [复制代码](#)

```
1 UIKit_EXTERN NSString *const CancelActivateSuccessNotification;
```

.m

ini

[代码解读](#) [复制代码](#)

```
1 NSString *const CancelActivateSuccessNotification = @"CancelActivateSuccessNotification";
```

Swift时代还是这么写吗？

Swift时代的通知名写法

其实Swift的早期，基本上还是沿用着OC的这一套写法来写通知名，不过在Swift4.2之后就迎来比较大的改变，让我们来看看调用的API与源码：

swift

[代码解读](#) [复制代码](#)

```
1 open func post(name aName: NSNotification.Name, object anObject: Any?)
2
3 open func post(name aName: NSNotification.Name, object anObject: Any?, userInfo aUserInfo: [AnyHash:
```

发通知的时候，通知名被一个 `NSNotification.Name` 类型代替了，我们进去追着 `NSNotification.Name` 看：

```
Foundation > NSNotification > NotificationCenter
1  import CoreFoundation
2
3  extension NSNotification {
4
5
6      public struct Name : Hashable, Equatable, RawRepresentable, @unchecked Sendable {
7
8          public init(_ rawValue: String)
9
10         public init(rawValue: String)
11     }
12 }
13
```

@稀土掘金技术社区

大家一定要记住这种编码的书写方式，先送上结论：

- 可以在一个类型里面再定义一个类型，大家可以自己尝试。
- 什么时候嵌套？为何要这么写？当嵌套的类型与外层定义的类型有着较强关联的时候可以这么写。

说完了这些，我们可以看到在Swift中，发通知，通知名不再是一个字符串了，而是一个 `NSNotification.Name` 类型了。

那么在开发过程中，我们如何使用呢？我们不妨还是从系统提供的API开始找：

```
438 extension UIApplication {
439     @available(iOS 4.0, *)
440     public class let didEnterBackgroundNotification: NSNotification.Name
441
442     public class let willEnterForegroundNotification: NSNotification.Name
443
444     public class let didFinishLaunchingNotification: NSNotification.Name
445
446     public class let didBecomeActiveNotification: NSNotification.Name
447
448     public class let willResignActiveNotification: NSNotification.Name
449
450     public class let didReceiveMemoryWarningNotification: NSNotification.Name
451
452     public class let willTerminateNotification: NSNotification.Name
453
454     public class let significantTimeChangeNotification: NSNotification.Name
455 }
456
```

@稀土掘金技术社区

因为Swift可以随处编写一个类的分类，于是在一个类的分类中定义好该类的通知名这种书写方式随处可见，这样的好处就是通知名与类紧紧联系在一起，一来便于查找，二来便于绑定业务类型。

php

代码解读 复制代码

```
1 NotificationCenter.default.post(name: UIApplication.didFinishLaunchingNotification, object: nil)
```

上面这个通知一发出，通过通知名我就知道是涉及 `UIApplication` 的操作行为。

说完了系统提供的API，我们再来看看一些知名第三方库是怎么定义呢，这里以 `Alamofire` 为例。

```
Pods > Pods > Alamofire > Notifications / No Selection
25 import Foundation
26
27 extension Request {
28     /// Posted when a 'Request' is resumed. The 'Notification' contains the resumed 'Request'.
29     public static let didResumeNotification = Notification.Name(rawValue:
30         "org.alamofire.notification.name.request.didResume")
31     /// Posted when a 'Request' is suspended. The 'Notification' contains the suspended 'Request'.
32     public static let didSuspendNotification = Notification.Name(rawValue:
33         "org.alamofire.notification.name.request.didSuspend")
34     /// Posted when a 'Request' is cancelled. The 'Notification' contains the cancelled 'Request'.
35     public static let didCancelNotification = Notification.Name(rawValue:
36         "org.alamofire.notification.name.request.didCancel")
37     /// Posted when a 'Request' is finished. The 'Notification' contains the completed 'Request'.
38     public static let didFinishNotification = Notification.Name(rawValue:
39         "org.alamofire.notification.name.request.didFinish")
40
41     /// Posted when a 'URLSessionTask' is resumed. The 'Notification' contains the 'Request' associated
42     'URLSessionTask'.
43     public static let didResumeTaskNotification = Notification.Name(rawValue:
44         "org.alamofire.notification.name.request.didResumeTask")
45     /// Posted when a 'URLSessionTask' is suspended. The 'Notification' contains the 'Request' associated
46     'URLSessionTask'.
47     public static let didSuspendTaskNotification = Notification.Name(rawValue:
48         "org.alamofire.notification.name.request.didSuspendTask")
49     /// Posted when a 'URLSessionTask' is cancelled. The 'Notification' contains the 'Request' associated
50     'URLSessionTask'.
51     public static let didCancelTaskNotification = Notification.Name(rawValue:
52         "org.alamofire.notification.name.request.didCancelTask")
53 }
```

```
43     /// Posted when a 'URLSessionTask' is completed. The 'Notification' contains the 'Request' associa
44     /// 'URLSessionTask'.
45     public static let didCompleteTaskNotification = Notification.Name(rawValue:
46         "org.alamofire.notification.name.request.didCompleteTask")
47     }
```

Alamofire 保持了和系统API一样的风格来定义通知名。

我们再来看看 Kingfisher：

```
22  // AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
23  // LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
24  // OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
25  // THE SOFTWARE.
26
27  #if os(macOS)
28  import AppKit
29  #else
30  import UIKit
31  #endif
32
33  extension Notification.Name {
34      /// This notification will be sent when the disk cache got cleaned either there are cached files expired or the
35      /// total size exceeding the max allowed size. The manually invoking of 'clearDiskCache' method will not trigger
36      /// this notification.
37      ///
38      /// The 'object' of this notification is the 'ImageCache' object which sends the notification.
39      /// A list of removed hashes (files) could be retrieved by accessing the array under
40      /// 'KingfisherDiskCacheCleanedHashKey' key in 'userInfo' of the notification object you received.
41      /// By checking the array, you could know the hash codes of files are removed.
42      public static let KingfisherDidCleanDiskCache =
43          Notification.Name("com.onevcat.Kingfisher.KingfisherDidCleanDiskCache")
44  }
```

Kingfisher 是在 NSNotification.Name 分类中扩展了通知名。

顺带说一下，我自己管理与编写通知名是这样的：

swift

[代码解读](#) [复制代码](#)

```
1  extension Notification.Name {
2      enum LoginService {
3          /// 退出
4          static let logoutNotification = NSNotification.Name("logoutNotification")
5      }
6  }
```

php

[代码解读](#) [复制代码](#)

```
1  NotificationCenter.default.post(name: .LoginService.logoutNotification, object: nil)
```

通过在 NSNotification.Name 分类中进行二级业务扩展，细化通知名。

至于大家更喜欢哪一种写法，那就是仁者见仁智者见智的事情了。

总结

本篇文章从 `NotificationCenter` 发通知的通知名开始，对OC到Swift的写法演进进行梳理与说明，举了系统API和著名第三方库的例子，给大家讲解如何写好并管理好 `NSNotification.Name`。

吐槽

掘金的这个编辑器，我直接从Xcode里面复制粘贴代码的体验真的很不友好，导致我比较长的代码都是截图，只有较少的代码使用的代码块。

自己写的项目，欢迎大家star★

[RxStudy](#): RxSwift/RxCocoa框架，MVVM模式编写wanandroid客户端。

[GetXStudy](#): 使用GetX，重构了Flutter wanandroid客户端。

标签: Objective-C Swift 掘金·金石计划 话题: 金石计划征文活动

本文收录于以下专栏



我的iOS开发

专栏目录

iOS原生开发的小谈，大部分会以Swift为主，偶尔来点OC。

275 订阅 · 110 篇文章

订阅

[上一篇](#) Swift: 使用 Decimal 接受金额并进... [下一篇](#) iOS开发: 关于路由

评论 2

- 

平等表达，友善交流

 

0 / 1000  发送
- 最热 最新



程序员劝退师 @百毒谷狗麻花藤科技有限公司

哈哈

1年前  点赞  评论 ...



跳入西湖 iOS

学习了，系统通知名的写法比宏更方便了，因为能够全局引用

1年前  点赞  评论 ...

为你推荐

用Swift5.1实现iOS中的远程推送流程

QiShare 5年前 4.9k 20 评论 iOS Swift

Swift 学习：从 Objective-C 到 Swift

sing1ee 9年前 3.7k 118 评论 iOS Objective... Swift

iOS swift2.3 迁移到3.0 遇到的一些问题

我的时代我来创 7年前 450 点赞 评论 Swift iOS

iOS源码解析: NotificationCenter是如何实现的?

超越杨超越 5年前 4.9k 12 6 iOS

iOS面试：OC基础 (二)

iOS小王 3年前 1.5k 4 评论 面试 iOS

一个才适应 Swift2.2 的开发者眼中的 Swift 3.0 和 iOS 10

amoywork 8年前 1.3k 26 评论 iOS Swift Xcode

Swift开发-NSNotificationCenter 写法

莱蕙 1年前 42 2 评论 Swift

iOS-Swift-从OC到Swift

cocoCola91667 2年前 163 点赞 评论 Swift

给即将失业的iOS从业者整理的OC基础知识

诗和远方14939562... 11月前 92 1 1 前端

一个才适应 Swift 2.2 的开发者眼中的 Swift 3.0 和 iOS 10

宇朋Look 8年前 920 36 评论 iOS

iOS面试题：基础知识 (二)

iOS沐橙君 3年前 509 2 评论 面试

XCode10 swift4.2 适配遇到的坑

Crazy凡 6年前 4.4k 3 评论 iOS Swift

Swift 3必看：foundation中数据引用类型改为值类型

独立开花卓富贵 8年前 771 19 2 Swift iOS

swift学习-02 从oc到swift01

路过看风景 4年前 462 点赞 评论 Swift

Swift 进阶 (十五) 从OC到Swift (上)

FunkyRay 4年前 789 3 评论 Swift

AI助手